MINI PROJECT REPORT

on

CAMPUS NAVIGATOR

A Graph-Based Campus Route and Network Planner

using Data Structures and Algorithms in C++

Subject: Advanced Data Structures and Algorithms (ADSA)

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide for the valuable guidance, constant encouragement and helpful suggestions throughout this project. I am also thankful to the Head of the Department and all the faculty members of the Department of Computer Science and Engineering for providing the facilities and the knowledge that made this work possible. Finally, I thank my family and friends for their support and motivation.

ABSTRACT

Finding the way around a large campus is a problem that every new student and visitor faces. This project, **Campus Navigator**, models a university campus as a weighted undirected graph in which buildings are vertices and walking roads are edges labelled with distances in metres. On top of this model, a menu-driven C++ application solves ten practical problems: showing the campus map, finding the shortest walking route (Dijkstra's algorithm with a priority queue), listing distances to all places, exploring the campus in BFS and DFS order, planning the cheapest cable or road network (Kruskal's minimum spanning tree with a Union-Find structure), detecting critical roads (bridges using DFS low-link values), keeping an alphabetical location directory (binary search tree) with fast name lookup (hash table with chaining and automatic rehashing), remembering the last five route searches (singly linked list), compressing announcements (Huffman coding) and planning the best visitor tour within a time limit (0/1 knapsack dynamic programming). Every data structure is written from scratch, without using ready-made containers for the core logic. The tested program finds the shortest route from Main Gate to Hostel as 700 m, builds a spanning network of 1350 m instead of 2710 m, and compresses a sample announcement by 48.8%.

Keywords: Graph, Dijkstra, Kruskal, Union-Find, Bridges, Huffman Coding, Knapsack, Hash Table, BST, C++

TABLE OF CONTENTS

Abstract 3
1. Introduction 5
2. Objectives and Syllabus Mapping 5
3. System Requirements 6
4. System Design 6
5. Data Structures Used 8
6. Algorithms Implemented 10
7. Complexity Analysis 11
8. Testing and Results 12
9. Conclusion and Future Scope 14
References 15

LIST OF FIGURES AND TABLES

Figure 4.1 Module structure of the project
Figure 4.2 Campus graph with MST roads and the bridge highlighted
Table 2.1 Menu options mapped to syllabus topics
Table 4.1 Locations and roads of the sample campus
Table 7.1 Time complexity of the main operations
Table 8.1 Test cases and observed results
Table 8.2 Order of finalisation in Dijkstra’s algorithm
Table 8.3 Kruskal’s algorithm on the sample campus

1. Introduction

1.1 Background

A university campus is a small city: dozens of buildings are connected by a network of roads and footpaths, and thousands of people move through it every day. New students, visiting parents and guests often lose time because they do not know the shortest way between two places. Campus administrators face related problems, such as deciding how to lay cables or water pipes at minimum cost, or finding which roads are so important that closing them would cut off an entire building.

All of these are classical problems of graph theory. If every building is treated as a node and every road as an edge carrying a distance, then route finding becomes a shortest-path problem, cable laying becomes a minimum spanning tree problem, and finding critical roads becomes a bridge-finding problem. Studying Advanced Data Structures and Algorithms is most effective when these ideas are seen working together on one realistic problem, which is the motivation behind this project.

1.2 Problem Statement

To design and implement a console application that stores a campus as a weighted graph and answers the questions a student or an administrator would ask: what is the shortest way from A to B, how can all buildings be connected at the least cost, which roads are critical, how can the data be searched and stored efficiently, and how can a short visit be planned in limited time. The application must be built on data structures and algorithms written from first principles so that their working can be explained and analysed.

1.3 Scope of the Project

- A sample campus of 10 locations and 14 roads is pre-loaded; more locations and roads can be added while the program is running.
- All distances are in metres and roads are bidirectional. A fixed walking speed of 80 metres per minute is used for time estimates.
- The program is text-based (console) and runs on any system with a C++17 compiler.
- The project concentrates on algorithm design and data-structure implementation rather than on a graphical map.

2. Objectives and Syllabus Mapping

The main objectives of the project are listed below.

1. To implement the core data structures of the syllabus (linked list, binary search tree, hash table with chaining, disjoint-set union) from scratch.
2. To apply graph algorithms (Dijkstra, BFS, DFS, Kruskal, bridge detection) to a real-world campus model.

3. To use a greedy technique (Huffman coding) and a dynamic-programming technique (0/1 knapsack) on practical campus tasks.
4. To analyse the time complexity of every operation and verify correctness through test runs.

Table 2.1 shows how each menu option of the program is linked to a topic of the course, so that the work can be traced directly to the syllabus.

Table 2.1 Menu options mapped to syllabus topics

Option	Feature	Topic / Technique
1	Show campus map	Graph representation (adjacency list)
2, 3	Shortest route; distances to all places	Dijkstra's algorithm, priority queue
4	Explore from a place	BFS and DFS traversal
5	Cheapest road / cable network	Kruskal's MST, Union-Find
6	Critical roads	Bridges using DFS low-link values
7	Location directory, name search	Binary search tree, hashing with chaining
8	Last 5 route searches	Singly linked list
9	Compress an announcement	Huffman coding (greedy)
10	Visitor tour planner	0/1 knapsack (dynamic programming)
11, 12	Add road / add location	Dynamic graph update, hash-table rehashing

3. System Requirements

Item	Requirement
Processor / RAM	Any modern processor, 1 GB RAM or more
Operating system	Windows, Linux or macOS
Compiler	g++ / MinGW / Code::Blocks with C++17 support
Build command	g++ -std=c++17 -o campus main.cpp
Input	Keyboard (menu number, location number or name; names are not case sensitive)
Output	Text on the console

The program has no external library or database dependency; the whole project consists of four source files (about 25 KB), which keeps it easy to compile, read and demonstrate.

4. System Design

4.1 Overall Architecture

The code is divided into four files. `ds.h` contains the basic data structures; `graph.h` builds the graph class on top of them; `huffman.h` contains the compression module; and `main.cpp`

holds the campus data, the menu, the input helpers and the knapsack planner. Figure 4.1 shows how the files depend on each other.

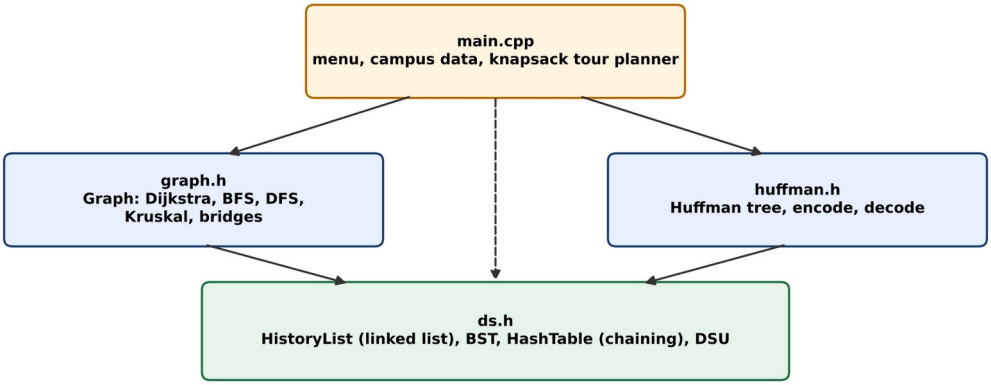


Figure 4.1 Module structure of the project

4.2 The Campus Model

The campus is stored as an adjacency list: `vector<vector<Edge>> adj`, where every `Edge` holds the destination vertex and the distance. A second list, `roads`, keeps each road once (as `u, v, w`) because Kruskal's algorithm needs a flat edge list. A hash table `index` maps a location name to its vertex number, so that a user can type either a number or a name. Figure 4.2 shows the sample campus used for all tests.

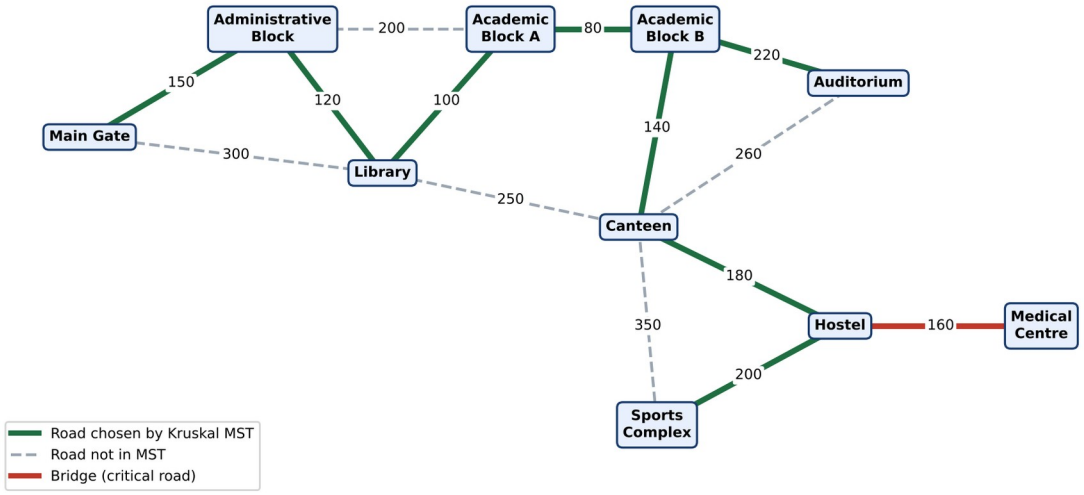


Figure 4.2 Campus graph with MST roads and the bridge highlighted

Table 4.1 Locations and roads of the sample campus

No.	Location	Connected to (distance in m)
1	Main Gate	Administrative Block (150), Library (300)
2	Administrative Block	Library (120), Academic Block A (200)
3	Library	Academic Block A (100), Canteen (250)
4	Academic Block A	Academic Block B (80)

No.	Location	Connected to (distance in m)
5	Academic Block B	Canteen (140), Auditorium (220)
6	Canteen	Hostel (180), Sports Complex (350), Auditorium (260)
7	Hostel	Sports Complex (200), Medical Centre (160)
8 - 10	Sports Complex, Auditorium, Medical Centre	Connected through the roads listed above

4.3 Module Responsibilities

- **Graph class** – adds locations and roads (rejecting self-loops, duplicates and non-positive lengths) and offers `dijkstra`, `bfs`, `dfs`, `kruskal` and `bridges`.
- **Menu layer** – `readLine`, `readInt` and `askLocation` validate input, so typing letters instead of numbers never crashes the program.
- **Huffman class** – builds a code tree from the text, encodes it to a bit string and decodes it back for verification.

4.4 Input Handling

A user may type either the location number or its name. The helper `askLocation` checks whether the input is made only of digits; if so it converts it to a zero-based index, otherwise it looks the name up in the hash table. It keeps asking until a valid location is entered, so a spelling mistake never stops the program.

```
int askLocation(const string &prompt) {
    while (true) {
        string s = readLine(prompt);
        if (cin.eof()) return -1;
        bool digits = !s.empty() && all_of(s.begin(), s.end(), ::isdigit);
        int id = digits ? stoi(s) - 1 : campus.index.get(s);   // hash-table lookup
        if (id >= 0 && id < campus.size()) return id;
        cout << "  Location not found. Type its number or exact name.\n";
    }
}
```

Listing 4.1 Accepting a location by number or by name

5. Data Structures Used

All four data structures below are implemented in `ds.h` without using library containers for their core logic (only `vector` and `list` are used as raw storage in the hash table).

5.1 Singly Linked List – Route History

The class `HistoryList` remembers the last five routes searched. A new entry is inserted at the front in O(1) time. When the size exceeds the capacity, the tail node is removed, so the list always shows the most recent search first.

```
void add(const string &s) {
    Node *n = new Node(s);
    n->next = head;  head = n;  size++;
    if (size > capacity) {            // drop the oldest entry (tail)
        Node *cur = head;
        while (cur->next->next) cur = cur->next;
        delete cur->next;  cur->next = nullptr;  size--;
    }
}
```

Listing 5.1 Insertion at the head with removal of the oldest node

5.2 Binary Search Tree – Location Directory

Every location name is inserted into a BST using case-insensitive comparison. An in-order traversal visits the keys in alphabetical order, which is used to print the directory. Duplicate names are ignored. For the ten sample names the tree height is 5.

5.3 Hash Table with Chaining – Name Lookup

The hash table converts a location name into its vertex number in O(1) average time. A polynomial rolling hash (multiplier 31) selects the bucket, and collisions are handled by chaining: each bucket is a `list` of key-value pairs. The table starts with 8 buckets and doubles its size whenever the load factor goes above 0.75, after which all keys are re-inserted (rehashing). After loading the ten locations the table has 16 buckets, a load factor of 0.62 and a longest chain of 2.

```
void put(const string &rawKey, int value) {
    string key = toLower(rawKey);
    auto &chain = table[hashOf(key)];
    for (auto &kv : chain)
        if (kv.first == key) { kv.second = value; return; }
    chain.push_back({key, value});
    count++;
    if ((double)count / table.size() > 0.75) rehash();
}
```

Listing 5.2 Insertion with chaining and automatic rehashing

5.4 Disjoint Set Union (Union-Find)

Kruskal’s algorithm must quickly decide whether two buildings are already connected. The `DSU` class answers this with `find` (using path compression) and `unite` (using union by rank), which together run in almost constant amortised time.

```
int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }
bool unite(int a, int b) {
    a = find(a);  b = find(b);
    if (a == b) return false;           // already connected -> would form a cycle
    if (rnk[a] < rnk[b]) swap(a, b);
    parent[b] = a;
    if (rnk[a] == rnk[b]) rnk[a]++;
    return true;
}
```

Listing 5.3 Union-Find with path compression and union by rank

6. Algorithms Implemented

6.1 Dijkstra’s Shortest Path

Dijkstra’s algorithm keeps a distance array initialised to infinity and a min-priority queue of (distance, vertex) pairs. The nearest unprocessed vertex is repeatedly removed from the queue and its neighbours are relaxed. Stale queue entries are skipped, and a `parent` array allows the path to be rebuilt from the destination back to the source. Because all road lengths are positive, the result is guaranteed optimal. Unreachable places keep the value INF and are reported as unreachable (-1).

```
while (!pq.empty()) {
    auto [d, u] = pq.top();  pq.pop();
    if (d > dist[u]) continue;                    // stale entry
    for (auto &e : adj[u])
        if (d + e.w < dist[e.to]) {               // relaxation
            dist[e.to] = d + e.w;
            parent[e.to] = u;
            pq.push({dist[e.to], e.to});
        }
}
```

Listing 6.1 Core loop of Dijkstra’s algorithm

6.2 BFS and DFS Traversal

BFS uses a queue and visits the nearest places first, in layers; it also gives the route with the fewest stops. DFS uses recursion and goes as deep as possible before backtracking. The program prints both orders from any chosen starting place so that the difference can be seen directly.

6.3 Kruskal’s Minimum Spanning Tree

All roads are sorted by length. They are then taken one by one, and a road is accepted only if `unite` reports that its two ends were in different components. The accepted roads form the cheapest network that connects every building, which is useful for laying cables or pipes. If fewer than V-1 roads are accepted, the program warns that the campus is not fully connected.

6.4 Bridge Detection

A bridge is a road whose removal disconnects the graph. The algorithm runs a DFS and stores for every vertex its discovery time `tin` and the lowest discovery time `low` reachable through back-edges. A tree edge (u, v) is a bridge when `low[v] > tin[u]`, meaning that no other route leads from v’s subtree back to u or above.

```
tin[u] = low[u] = ++timer;
for (auto &e : adj[u]) {
    if (e.to == parent) continue;
    if (tin[e.to]) low[u] = min(low[u], tin[e.to]);     // back-edge
    else {
        bridgeDfs(e.to, u, timer, tin, low, out);
        low[u] = min(low[u], low[e.to]);
        if (low[e.to] > tin[u]) out.push_back({u, e.to, e.w});   // bridge
    }
}
```

Listing 6.2 Bridge detection with low-link values

6.5 Huffman Coding

A frequency table of the characters in an announcement is built, and one tree node per character is placed in a priority queue. The two least frequent nodes are repeatedly merged into a parent node until a single tree remains. Moving left appends bit 0 and moving right appends bit 1, so frequent characters receive short codes. The encoded bits are decoded again and compared with the original text, and the program prints “[verified]” when both are identical.

6.6 0/1 Knapsack – Visitor Tour Planner

A visitor has a limited number of minutes and wants the most useful tour. Each attraction has a visiting time (weight) and a rating from 1 to 10 (value). The table `dp[i][t]` holds the best total rating using the first i attractions in t minutes: either the attraction is skipped (`dp[i-1][t]`) or taken (`dp[i-1][t-time] + rating`). Walking back through the table recovers the chosen attractions. Each place can be visited at most once, which is exactly the 0/1 condition.

```
for (int i = 1; i <= n; i++)
    for (int t = 0; t <= budget; t++) {
        dp[i][t] = dp[i - 1][t];                                    // skip
        if (attractions[i - 1].minutes <= t)                        // take
            dp[i][t] = max(dp[i][t], dp[i - 1][t - wt] + attractions[i - 1].rating);
        // wt = attractions[i - 1].minutes
    }
```

Listing 6.3 Knapsack table construction

7. Complexity Analysis

Table 7.1 Time complexity of the main operations

Operation	Time complexity	Remark
Dijkstra (binary heap)	O((V + E) log V)	Positive edge weights only
BFS / DFS / Bridges	O(V + E)	One visit per vertex and edge
Kruskal’s MST	O(E log E)	Dominated by sorting the roads
Union-Find find / unite	Almost O(1)	Path compression + union by rank
Huffman build	O(n log n)	n = distinct characters
Knapsack tour	O(items x minutes)	Pseudo-polynomial
BST insert (average)	O(log n)	Worst case O(n) if names arrive sorted
Hash-table lookup (average)	O(1)	Load factor kept below 0.75

Operation	Time complexity	Remark
Linked-list history insert	O(1) at front	Removing the tail takes O(capacity)

8. Testing and Results

The program was compiled with `g++ -std=c++17` and executed with the sample campus. Each feature was tested with known inputs, and the output was checked by hand against the graph in Figure 4.2. Table 8.1 summarises the main test cases.

Table 8.1 Test cases and observed results

Test	Input	Observed output	Result
Shortest route	Main Gate to Hostel	700 m via Admin Block, Library, Canteen	Pass
Distances	Source: Main Gate	Sports Complex 870 m is the farthest	Pass
MST	Option 5	9 roads, 1350 m (all roads 2710 m)	Pass
Bridges	Option 6	Only Hostel - Medical Centre	Pass
Hash statistics	Option 7	16 buckets, load 0.62, longest chain 2	Pass
Huffman	Exam announcement	600 bits to 307 bits, 48.8% saved, verified	Pass
Knapsack	120 minutes	Rating 24: Canteen, Library, Medical, Admin	Pass
Bad input	Letters in a number field	Message shown, program continues	Pass

8.1 Sample Output

```
Choose: 2
From (number or name): Main Gate
To   (number or name): Hostel
Shortest route (700 m, ~9 min walking):
  Main Gate -> Administrative Block -> Library -> Canteen -> Hostel

Choose: 5
Cheapest set of roads/cables that still connects every building (Kruskal + DSU):
  Academic Block A <-> Academic Block B  (80 m)
  Library <-> Academic Block A  (100 m)
  Administrative Block <-> Library  (120 m)
  ...
Total length needed: 1350 m (all roads together: 2710 m, saving 1360 m)

Choose: 6
Critical roads (closing any of them cuts part of the campus off):
  Hostel <-> Medical Centre (160 m)
```

Output 8.1 Route, spanning network and critical roads

```
Choose: 4
Start location: Library
BFS (nearest places first):
  Library -> Main Gate -> Administrative Block -> Academic Block A -> Canteen -> ...
DFS (go as deep as possible first):
  Library -> Main Gate -> Administrative Block -> Academic Block A -> Academic Block B
-> ...

Choose: 10
How many minutes do you have for the visit tour? 120
Best tour for 120 minutes (0/1 Knapsack DP) - total rating 24:
  - Medical Centre (25 min, rating 4)
  - Administrative Block (20 min, rating 3)
  - Canteen (45 min, rating 9)
  - Library (30 min, rating 8)
Time used: 120 min
```

Output 8.2 Traversals and tour planning

8.2 Worked Example: Dijkstra from Main Gate

Table 8.2 lists the order in which Dijkstra's algorithm finalises the vertices when started at Main Gate. The predecessor column is the `parent` array; following it backwards from Hostel gives Hostel ← Canteen ← Library ← Administrative Block ← Main Gate, which is the route printed by the program.

Table 8.2 Order of finalisation in Dijkstra's algorithm (source: Main Gate)

Step	Vertex finalised	Distance (m)	Predecessor
1	Main Gate	0	-
2	Administrative Block	150	Main Gate
3	Library	270	Administrative Block
4	Academic Block A	350	Administrative Block
5	Academic Block B	430	Academic Block A
6	Canteen	520	Library
7	Auditorium	650	Academic Block B
8	Hostel	700	Canteen
9	Medical Centre	860	Hostel
10	Sports Complex	870	Canteen

8.3 Worked Example: Kruskal's Algorithm

Table 8.3 shows every road in sorted order and whether Union-Find accepted it. Nine of the fourteen roads are accepted (V - 1 = 9), and the total accepted length is 1350 m. Roads that would close a cycle are rejected.

Table 8.3 Kruskal's algorithm on the sample campus

Road	Length (m)	Decision
Academic Block A - Academic Block B	80	Accept
Library - Academic Block A	100	Accept

Road	Length (m)	Decision
Administrative Block - Library	120	Accept
Academic Block B - Canteen	140	Accept
Main Gate - Administrative Block	150	Accept
Hostel - Medical Centre	160	Accept
Canteen - Hostel	180	Accept
Administrative Block - Academic Block A	200	Reject (cycle)
Hostel - Sports Complex	200	Accept
Academic Block B - Auditorium	220	Accept
Library - Canteen	250	Reject (cycle)
Auditorium - Canteen	260	Reject (cycle)
Main Gate - Library	300	Reject (cycle)
Canteen - Sports Complex	350	Reject (cycle)

8.4 Discussion of Results

A simple BFS from Main Gate to Hostel would choose Main Gate → Library → Canteen → Hostel, which has the fewest roads (3) but is 730 m long. Dijkstra's algorithm chooses a route with four roads that is only 700 m, because the two short roads through the Administrative Block (150 m + 120 m) beat the single 300 m direct road to the Library. This shows why a distance-aware algorithm is needed. The MST saves 1360 m, almost half of the total road length, and the bridge search correctly finds that the Medical Centre is reachable only through the Hostel, which is an important observation for emergency planning. For the announcement "Attention students: the library will remain open till midnight during exams", Huffman coding reduced 600 bits to 307 bits.

9. Conclusion and Future Scope

The Campus Navigator shows that one realistic problem can bring the whole syllabus together: graphs for the map, a priority queue for shortest paths, Union-Find for network planning, DFS for critical roads, hashing and trees for storage and search, a linked list for history, a greedy method for compression and dynamic programming for planning. Writing every structure from scratch gave a clear understanding of how operations work and what they cost.

9.1 Limitations

- The BST is not self-balancing, so names inserted in sorted order would make it degenerate into a list.
- The campus is hard-coded in the source file and is lost when the program exits.

- The compression ratio counts only the encoded bits, not the code table that a real decoder would need.

9.2 Future Scope

- Read and save the map from a file, and draw it graphically with SFML or Python.
- Replace the BST with an AVL or red-black tree; add A* search and a Prim's algorithm comparison.
- Use different speeds for different roads and add one-way or closed roads.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[2] E. Horowitz, S. Sahni and S. Rajasekaran, Fundamentals of Computer Algorithms, 2nd ed., Universities Press, 2008.

[3] M. A. Weiss, Data Structures and Algorithm Analysis in C++, 4th ed., Pearson, 2013.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011.

[5] ISO/IEC 14882:2017, Programming Language C++ (C++17) – cppreference.com documentation.